



UNIVERSITY OF GENEVA
Department Informatic

Software Modeling and Verification

14XBlabla

Michel Jean Joseph Donnet

October 8, 2025

Table of Contents

1	Preface	2
2	Introduction	2
3	Formal language, semantics and notation	2
3.1	Syntax	2
3.2	Big-step semantics (memory transformer)	3
4	Hoare logic: triples and rules	3
4.1	Axioms and inference rules	4
5	Weakest precondition calculus (WP)	4
5.1	WP rules	4
5.2	WP correctness theorem (loop-free programs)	5
6	Soundness and completeness	5
6.1	Soundness (informal)	5
6.2	Relative completeness (Cook's theorem)	6
7	Formal proofs	6
7.1	Soundness of Hoare logic (full proof)	6
7.2	WP-correctness (formal proof)	7
8	Termination and total correctness	7
9	Semantic execution examples	8
9.1	Example A — simple sequence	8
9.2	Example B — conditional with memory states	9
9.3	Example C — loop unfolding (addition)	9
10	Extended formal verification examples	10
10.1	Addition by repeated increment (full proof)	10
10.2	Factorial	10
10.3	Euclid's algorithm (GCD)	11
10.4	Binary search (formal invariants)	11
10.5	Nested conditionals — semantic development	11
10.6	WP with quantifiers	12
11	Recap	12

1 Preface

These notes present an academic treatment of software modeling and verification. Program states (memories) are denoted by the Greek letter μ . Satisfaction of an assertion P by a state μ is written $\mu \models P$.

Inline mathematical expressions use single dollar signs $\$ \dots \$$; display formulas use $\$ \$ \dots \$ \$$ where appropriate.

This document focuses on:

- Deductive verification using Hoare logic
 - Weakest preconditions (WP) and their computation
 - Loop invariants and variants
 - Soundness and completeness proofs
 - Semantic execution examples using μ and Mem
-

2 Introduction

Formal verification provides mathematical guarantees that programs meet their specifications. It complements testing by proving properties for all inputs using logical reasoning and program semantics.

These notes cover the formal foundations of program verification and illustrate their use through many examples and formal proofs.

3 Formal language, semantics and notation

3.1 Syntax

We consider a classical imperative core language:

- **Arithmetic expressions** A : variables x , integer constants c , binary operators $+$, $-$, $*$, and function calls.
- **Boolean expressions** B : comparisons $A_1 \leq A_2$, $A_1 = A_2$, logical connectives $\wedge$, $\vee$, $\neg$.
- **Instructions** I :
 - Assignment $x := E$
 - Sequencing $I_1; I_2$
 - Conditional `if b then I_t else I_e endif`

– Loop **while** b **do** I **done**

A program state (memory) is a total function $\mu : \text{Var} \rightarrow \mathbf{Z}$.

We write $\mu[x \leftarrow v]$ for the state identical to μ except x maps to v .

The evaluation function $\text{Val}(e, \mu)$ yields the value of expression e under μ .

3.2 Big-step semantics (memory transformer)

Define the partial function $\text{Mem}(I, \mu)$ returning the final state after executing I from state μ if I terminates; otherwise it is undefined.

- **Assignment**

$$\text{Mem}(x := t, \mu) = \mu[x \leftarrow \text{Val}(t, \mu)].$$

- **Sequence**

$$\text{Mem}(I_1; I_2, \mu) = \text{Mem}(I_2, \text{Mem}(I_1, \mu)) \text{ (when defined).}$$

- **Conditional**

$$\begin{aligned} \text{Mem}(\text{if } b \text{ then } I_t \text{ else } I_e, \mu) = \\ \text{if } \text{Val}(b, \mu) = \text{true} \text{ then } \text{Mem}(I_t, \mu) \text{ else } \text{Mem}(I_e, \mu). \end{aligned}$$

- **While loop**

$\text{Mem}(\text{while } b \text{ do } I \text{ done}, \mu)$ is defined recursively:

- If $\text{Val}(b, \mu) = \text{false}$, return μ .
- Else let $\nu = \text{Mem}(I, \mu)$ (if defined) and continue from ν .

Execution is deterministic when defined.

4 Hoare logic: triples and rules

A Hoare triple has the form $\{P\} I \{Q\}$, where P and Q are assertions in first-order logic.

Semantics: the triple is valid iff for all states μ ,

- if $\mu \models P$ and
- if $\text{Mem}(I, \mu)$ is defined, then
- $\text{Mem}(I, \mu) \models Q$.

4.1 Axioms and inference rules

- **Assignment axiom**

$$\{Q[x \leftarrow t]\} \ x := t \ \{Q\}$$

- **Consequence rule**

If $P' \Rightarrow P$, $\{P\}I\{Q\}$, and $Q \Rightarrow Q'$, then $\{P'\}I\{Q'\}$.

- **Sequence rule**

From $\{P\}I_1\{R\}$ and $\{R\}I_2\{Q\}$ infer $\{P\}I_1; I_2\{Q\}$.

- **Conditional rule**

From $\{P \wedge b\}I_t\{Q\}$ and $\{P \wedge \neg b\}I_e\{Q\}$ infer
 $\{P\} \text{ if } b \text{ then } I_t \text{ else } I_e \text{ endif } \{Q\}$.

- **While rule**

If there exists an invariant J such that $\{J \wedge b\}I_{body}\{J\}$ and $J \wedge \neg b \Rightarrow Q$, then
 $\{J\} \text{ while } b \text{ do } I_{body} \text{ done } \{Q\}$.

To use from a precondition P , also prove $P \Rightarrow J$.

These rules address **partial correctness**. Total correctness additionally requires termination proofs.

5 Weakest precondition calculus (WP)

For a postcondition Q , the weakest precondition $WP(I, Q)$ is the strongest assertion P such that $\{P\}I\{Q\}$ holds.

5.1 WP rules

- **Assignment**

$$WP(x := t, Q) = Q[x \leftarrow t]$$

- **Sequence**

$$WP(I_1; I_2, Q) = WP(I_1, WP(I_2, Q))$$

- **Conditional**

$$WP(\text{if } b \text{ then } I_t \text{ else } I_e, Q) = (b \Rightarrow WP(I_t, Q)) \wedge (\neg b \Rightarrow WP(I_e, Q))$$

- **While (with invariant)**

For a loop `while b do I done` and a candidate invariant J , set

$$WP(\text{while } b \text{ do } I \text{ done}, Q) := J$$

provided that the following verification conditions hold:

- **Initialization:** $P \Rightarrow J$
- **Preservation:** $J \wedge b \Rightarrow WP(I, J)$
- **Exit condition:** $J \wedge \neg b \Rightarrow Q$

In practice, these verification conditions (VCs) are discharged using SMT solvers.

5.2 WP correctness theorem (loop-free programs)

Theorem. For loop-free instructions I , the Hoare triple $\{P\}I\{Q\}$ is valid if and only if

$$P \Rightarrow WP(I, Q).$$

Proof sketch.

By structural induction on I :

- **Assignment:** $\{Q[x \leftarrow t]\}x := t\{Q\}$ holds by substitution semantics.
- **Sequence:** Induction applies to each part of the sequence.
- **Conditional:** The WP is exactly the conjunction of implications for both branches.

□

6 Soundness and completeness

6.1 Soundness (informal)

Any Hoare triple derived using the axioms and inference rules is semantically valid.

Soundness ensures the proof system never derives incorrect judgments.

The proof is by induction on the structure of derivations. For the while-rule, a finite induction on the number of loop iterations is used.

6.2 Relative completeness (Cook's theorem)

Theorem (Cook, 1978).

If the assertion language is expressive enough (i.e., can express strongest postconditions and necessary arithmetic facts) and all valid first-order formulas are provable, then Hoare logic is relatively complete: every semantically valid triple is derivable.

Sketch.

- Translate the program into first-order verification conditions using strongest postconditions or weakest preconditions.
- Use expressivity of the logic to state required invariants.
- Completeness is *relative* because it depends on the power of the underlying logic (e.g., arithmetic is undecidable in general).

This theorem provides a theoretical foundation: given strong enough invariants, Hoare logic can prove any valid partial correctness property.

7 Formal proofs

7.1 Soundness of Hoare logic (full proof)

Theorem (Soundness).

If $\{P\}I\{Q\}$ is derivable in Hoare logic, then for all states μ :

- if $\mu \models P$ and
- if $\text{Mem}(I, \mu)$ is defined, then
- $\text{Mem}(I, \mu) \models Q$.

Proof.

By induction on the derivation of $\{P\}I\{Q\}$.

- **Assignment axiom:** Let I be $x := t$ and assume $\{Q[x \leftarrow t]\}x := t\{Q\}$.
Suppose $\mu \models Q[x \leftarrow t]$.
By semantics of substitution: $\mu[x \leftarrow \text{Val}(t, \mu)] \models Q$.
But $\text{Mem}(x := t, \mu) = \mu[x \leftarrow \text{Val}(t, \mu)]$.
Therefore the triple is valid.
- **Consequence rule:** Straightforward from logical implication.
- **Sequence rule:** Induction hypothesis (IH) for $\{P\}I_1\{R\}$ and $\{R\}I_2\{Q\}$. Combine using definition of $\text{Mem}(I_1; I_2, \mu)$.

- **Conditional rule:** Apply IH to each branch depending on evaluation of b .
- **While rule:** Use induction on the number n of loop iterations.
 Define states $\mu_0, \mu_1, \dots, \mu_n$ where $\mu_{i+1} = \text{Mem}(I_{\text{body}}, \mu_i)$.
 Show by induction that the invariant J holds for all μ_i .
 When the loop exits (b false), $J \wedge \neg b \Rightarrow Q$ implies the postcondition.

□

7.2 WP-correctness (formal proof)

Theorem.

For loop-free I , for all states μ :

$$\mu \models WP(I, Q) \quad \text{iff} \quad \text{Mem}(I, \mu) \text{ defined} \Rightarrow \text{Mem}(I, \mu) \models Q.$$

Proof.

By structural induction on I :

- **Assignment:**
 $\mu \models WP(x := t, Q)$ iff $\mu \models Q[x \leftarrow t]$
 iff $\mu[x \leftarrow \text{Val}(t, \mu)] \models Q$
 iff $\text{Mem}(x := t, \mu) \models Q$.
- **Sequence:**
 Induction hypothesis on I_2 gives equivalence for $WP(I_2, Q)$; then apply IH on I_1 .
- **Conditional:**
 Split on $\text{Val}(b, \mu)$; apply IH on the appropriate branch.

□

8 Termination and total correctness

Partial correctness makes no claim about whether a program halts.

To prove **total correctness**, termination must also be shown. The standard method is to exhibit a **variant** $V(\mu)$ taking values in $\mathbb{N}$, such that:

1. $J(\mu)$ implies $V(\mu) \geq 0$.
2. $J(\mu) \wedge b(\mu)$ implies $V(\text{Mem}(I_{\text{body}}, \mu)) < V(\mu)$.

Since there is no infinite descending chain in $\mathbb{N}$, this guarantees that the loop must eventually terminate.

Examples of variants:

- For a countdown loop: $V(\mu) = n - i$.
- For a factorial loop: $V(\mu) = n - (i - 1)$.
- For Euclid's algorithm: $V(\mu) = b$.

Lemma (Termination).

Under the above conditions, every execution starting from a state satisfying J terminates.

Proof.

Assume an infinite execution. Then we obtain an infinite strictly decreasing sequence $V(\mu_0) > V(\mu_1) > \dots$ in $\mathbb{N}$, which is impossible.

□

9 Semantic execution examples

These examples illustrate how program execution transforms states μ step by step, and how WP calculations correspond to this semantics.

9.1 Example A — simple sequence

Program:

```
x := 1;
y := x + 2;
```

Let the initial state be μ_0 with $\mu_0(x) = 0$ and $\mu_0(y) = 0$.

1. Assignment $x := 1$:

$$\text{Val}(1, \mu_0) = 1 \text{ and } \mu_1 = \mu_0[x \leftarrow 1].$$

2. Assignment $y := x + 2$:

$$\text{Val}(x + 2, \mu_1) = \mu_1(x) + 2 = 3 \text{ and } \mu_2 = \mu_1[y \leftarrow 3].$$

The final state μ_2 has $x = 1$, $y = 3$.

WP check:

Postcondition $Q : y = 3$

- $WP(y := x + 2, Q) = (x + 2 = 3)$
- $WP(x := 1, x + 2 = 3) = (1 + 2 = 3)$ which is true.

So the program ensures $y = 3$ for all initial states.

9.2 Example B — conditional with memory states

Program:

```
if x > 0 then
  y := x
else
  y := -x
fi
```

Initial state $\mu(x) = -2$.

- Evaluate guard: $\text{Val}(x > 0, \mu) = \text{false}$.
- Execute else-branch: $\text{Val}(-x, \mu) = 2$.
- $\mu' = \mu[y \leftarrow 2]$.

Postcondition $y \geq 0$ holds in μ' .

9.3 Example C — loop unfolding (addition)

Program:

```
res := a;
aux := b;
while aux > 0 do
  res := res + 1;
  aux := aux - 1;
done
```

Initial state: $\mu_0(a) = 2, \mu_0(b) = 2, \mu_0(res) = 0, \mu_0(aux) = 0$.

- After initialization: $\mu_1 = \mu_0[res \leftarrow 2][aux \leftarrow 2]$.
- Iteration 1: $aux = 2 > 0$,
 $\mu_2 = \mu_1[res \leftarrow 3][aux \leftarrow 1]$.
- Iteration 2: $aux = 1 > 0$,
 $\mu_3 = \mu_2[res \leftarrow 4][aux \leftarrow 0]$.
- Exit: guard false, final μ_3 has $res = 4$ and $aux = 0$.

Invariant $res + aux = a + b$ is preserved across μ_1, μ_2, μ_3 .

10 Extended formal verification examples

10.1 Addition by repeated increment (full proof)

Program:

```
res := a;
aux := b;
while aux > 0 do
  res := res + 1;
  aux := aux - 1;
done
```

Precondition $b \geq 0$, postcondition $res = a + b$.

Invariant candidate:

$$J(\mu) \equiv res + aux = a + b \wedge aux \geq 0.$$

- **Initialization:** After assignments, J holds.
- **Preservation:** $J \wedge aux > 0 \Rightarrow WP(\text{body}, J)$ holds by substitution.
- **Exit:** $aux = 0$ gives $res = a + b$.

Variant $V(\mu) = aux$ decreases each iteration and is bounded below by 0 → termination.

10.2 Factorial

Program:

```
res := 1;
i := 1;
while i <= n do
  res := res * i;
  i := i + 1;
done
```

Invariant:

$$J(\mu) : 1 \leq i \leq n + 1 \wedge res = (i - 1)!.$$

- **Initialization:** holds after first two assignments.
- **Preservation:** multiplying by i updates res to $i!$ and increments i .
- **Exit:** $i = n + 1$ gives $res = n!$.

Variant $V(\mu) = n - (i - 1)$ decreases monotonically → termination.

10.3 Euclid's algorithm (GCD)

Program:

```
while b != 0 do
  r := a mod b;
  a := b;
  b := r;
done
return a
```

Invariant:

$$J(\mu) : \text{gcd}(a, b) = \text{gcd}(a_0, b_0) \wedge a \geq 0 \wedge b \geq 0.$$

Lemma: For $b > 0$, $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$ and $0 \leq a \bmod b < b$.

- **Preservation:** applying the lemma maintains gcd.
- **Exit:** when $b = 0$, $\text{gcd}(a, 0) = a$ gives postcondition.
- **Termination:** $V(\mu) = b$ decreases because b is replaced by $a \bmod b < b$.

10.4 Binary search (formal invariants)

Loop invariants track the interval $[lo, hi]$ where the target may lie:

- Before each iteration: if the key is in the array, it must be in $[lo, hi]$.
- The interval shrinks each iteration, guaranteeing termination.

The mid computation $lo + (hi - lo)/2$ requires reasoning over integer division. Exit conditions prove correctness for both “found” and “not found” cases.

10.5 Nested conditionals — semantic development

Program:

```
if x >= 0 then
  if x = 0 then y := 0 else y := 1 fi
else
  y := -1
fi
```

For $\mu(x) = -3$, the outer guard is false executes else sets $y = -1$.
 Postcondition $y \geq -1$ holds.
 WP calculation confirms the triple is valid for all x .

10.6 WP with quantifiers

Program:

```
i := 0;
found := false;
while i < N do
  if A[i] = key then found := true; break; fi;
  i := i + 1;
done
```

Postcondition:

$$found \Leftrightarrow \exists j < N. A[j] = key.$$

Loop invariant:

- If `found` is true, then some processed index $j < i$ satisfies $A[j] = key$.
- If `found` is false, then no index in $[0, i)$ matches.

VCs involve quantified formulas over indices. SMT solvers can often handle such VCs if the array theory is supported.

11 Recap

- **States:** μ represents memory; $\mu \models P$ means P holds in μ .
- **Hoare triple:** $\{P\}I\{Q\}$ asserts partial correctness.
- **WP calculus:** mechanical computation of necessary preconditions.
- **Loops:** require user-supplied invariants J and variants V for total correctness.
- **Proof obligations:**
 - Initialization: $P \Rightarrow J$
 - Preservation: $J \wedge b \Rightarrow WP(\text{body}, J)$
 - Exit: $J \wedge \neg b \Rightarrow Q$
 - Termination: V decreases and is bounded below

- **Tools:** Why3 generates VCs; SMT solvers discharge them automatically in many cases.
 - **Theory:** Hoare logic is sound and relatively complete.
-